

Universidade Federal do Ceará
Campus Quixadá
QXD0068 – Reuso de Software
Prof. Francisco Victor da Silva Pinheiro

Trabalho 1
Refatoração com Padrões de Projeto em Sistema Legado: Sistema de Monitoramento de Água

Maria Eduarda Almeida Rodrigues
Nathalia de Oliveira Lima

Quixadá – CE
2026

Sumário

1. Introdução
2. Caracterização e Diagnóstico do Sistema Legado
3. Padrões de Projeto Selecionados
4. Refatorações Realizadas
5. Análise dos Resultados
6. Conclusão
7. Referências

1. Introdução

A reutilização de software consiste em aproveitar soluções, componentes e comportamentos existentes para reduzir retrabalho e favorecer consistência. Em sistemas legados, a refatoração reorganiza a estrutura interna do código com a intenção de preservar o comportamento funcional observado. Os padrões de projeto GoF descrevem soluções recorrentes para problemas de criação, comunicação e organização de objetos.

Este trabalho utiliza como objeto de estudo o Water Monitoring System, projeto de código aberto publicado por Vinit Shahdeo. O sistema apresenta uma interface web para visualizar níveis percentuais de água em quatro reservatórios. Os níveis são representados graficamente e associados a estados textuais e cores, permitindo demonstrar o comportamento de um monitoramento simples.

O projeto foi escolhido por possuir escopo compreensível para uma atividade acadêmica de refatoração, mas conter responsabilidades que podem ser separadas: leitura dos níveis, configuração dos reservatórios, classificação do estado e atualização da interface. A proposta não é adicionar funcionalidades de monitoramento físico ou sensores reais, mas melhorar a organização do código existente.

O objetivo geral é aplicar quatro padrões GoF — Strategy, Factory, Observer e Facade — e analisar como a separação de responsabilidades pode favorecer manutenção, extensibilidade e reutilização. Os objetivos específicos são identificar problemas de projeto, justificar cada padrão, registrar as mudanças e definir procedimentos para validar a aplicação.

Repositório do projeto original: https://github.com/vinitshahdeo/water-monitoring-system
Demonstração publicada: https://vinitshahdeo.github.io/Water-Monitoring-System/

2. Caracterização e Diagnóstico do Sistema Legado

2.1 Caracterização do Sistema

O sistema tem como finalidade ilustrar níveis de água de diferentes reservatórios em uma interface web. A página principal apresenta elementos gráficos de nível, rótulos de estado e valores que podem ser informados por parâmetros na URL. Na versão refatorada analisada nesta documentação, a configuração de cada reservatório é obtida por WaterTankFactory, a classificação do nível é delegada a WaterLevelStrategy, os eventos são comunicados por WaterMonitoringObserver e a coordenação dessas partes é concentrada em WaterMonitoringFacade.

As tecnologias visíveis nos arquivos fornecidos incluem HTML, CSS e JavaScript no navegador, jQuery e o plugin createWaterBall-jquery.js para a representação gráfica dos reservatórios. A implementação apresentada usa funções e objetos JavaScript, sem exigir um processo de compilação para abrir a página estática.

Estrutura relevante após a refatoração: src/home.html; src/js/level-strategy.js; src/js/tank-factory.js; src/js/monitoring-observer.js; src/js/monitoring-facade.js; e a biblioteca gráfica em src/lib/createWaterBall-jquery.js. A presença exata desses arquivos deve ser confirmada no commit final.

2.2 Problemas Identificados

Na estrutura original, a inicialização dos reservatórios e a atualização da interface estavam concentradas em código JavaScript embutido na página. Esse formato tende a misturar leitura de parâmetros, definição de valores, classificação de status e manipulação do DOM. Quando essas tarefas ficam no mesmo bloco, uma mudança em uma regra pode exigir alterações na tela e dificulta reutilizar a lógica em outro contexto.

Os principais pontos de atenção considerados foram:
(a) responsabilidades concentradas na página;
(b) criação e configuração dos reservatórios acopladas à interface;
(c) regras de classificação pouco isoladas;
(d) ausência de um mecanismo explícito de notificação entre produtor de atualização e componentes interessados; e
(e) falta de uma interface simplificada para coordenar os subsistemas.

Evidência da versão refatorada compartilhada: em home.html, a inicialização chama WaterTankFactory.create(...), em seguida WaterLevelStrategy.classify(...), e atualiza elementos da página. A Facade foi criada para reunir a coordenação, enquanto o Observer permite notificar inscritos. Esses elementos evidenciam as responsabilidades que foram extraídas. Como o arquivo original completo não foi anexado neste relatório, a descrição dos problemas originais é estrutural e deve ser conferida contra a versão original arquivada no repositório.

3. Padrões de Projeto Selecionados

3.1 Strategy - padrão comportamental

- **Problema identificado:** regras de classificação misturadas à inicialização visual dificultam trocar limites ou acrescentar políticas sem alterar a interface.
- **Padrão aplicado:** Strategy.
- **Justificativa:** encapsula um algoritmo atrás de uma operação comum.
- **Componente:** WaterLevelStrategy, em src/js/level-strategy.js.
- **Benefício:** centralização da regra, teste isolado e possibilidade de acrescentar estratégias.

A implementação fornecida possui uma estratégia standard. Para níveis abaixo de 30 ou a partir de 90, retorna Crítico; para níveis abaixo de 60 ou a partir de 80, retorna Atenção; nos demais casos, retorna Seguro. A função também valida se o nível é numérico e está entre 0 e 100. Embora a estrutura aceite um nome de estratégia, o trecho compartilhado define somente standard.

3.2 Factory - padrão criacional

- **Problema identificado:** dados de configuração dos tanques, seletores e valores padrão podem ficar espalhados pela página.
- **Padrão aplicado:** Factory.
- **Justificativa:** centraliza a criação de um objeto de configuração e suas regras de validação.
- **Componente:** WaterTankFactory, em src/js/tank-factory.js.
- **Benefício:** configuração uniforme, menos repetição no cliente e local único para ajustar a forma de obter os dados.

A operação create(tankNumber, search) valida se o número está entre 1 e 4 e retorna um objeto com number, selector, statusSelector e level. O nível é obtido de parâmetros como tank1=65; quando ausente ou inválido, o código utiliza 50, e limita o valor numérico ao intervalo de 0 a 100.

3.3 Observer - padrão comportamental

- **Problema identificado:** os componentes interessados em atualizações não devem depender diretamente do produtor nem ser conhecidos individualmente por ele.
- **Padrão aplicado:** Observer.
- **Justificativa:** estabelece inscrição e notificação de ouvintes.
- **Componente:** WaterMonitoringObserver, em src/js/monitoring-observer.js.
- **Benefício:** comunicação menos acoplada e possibilidade de adicionar consumidores.

A implementação compartilhada oferece subscribe(listener) e notify(event). O método notify percorre uma cópia da lista de ouvintes, o que evita que alterações na lista durante a notificação afetem diretamente a iteração atual. Para comprovar o padrão integrado, a aplicação final deve criar o Observer, inscrever pelo menos um ouvinte e chamar notify durante o fluxo real.

3.4 Facade - padrão estrutural

- **Problema identificado:** o cliente precisa conhecer e coordenar vários módulos e a sequência de operações.
- **Padrão aplicado:** Facade.
- **Justificativa:** oferece uma interface simplificada para um conjunto de componentes.
- **Componente:** WaterMonitoringFacade, em src/js/monitoring-facade.js.
- **Benefício:** reduz o conhecimento exigido da página e concentra a coordenação.

O construtor recebe Factory, Strategy, Observer e a query string. O método initializeTank cria a configuração, classifica o nível e envia um evento com número, nível, seletores e status. A Facade estará efetivamente aplicada à interface somente se home.html carregar o arquivo, instanciar a fachada com as dependências, registrar o ouvinte que atualiza o DOM e chamar initializeTank para os tanques.

4. Refatorações Realizadas

4.1 Padrão 1 – Strategy

- **Situação original:** a classificação do nível podia estar junto ao código de inicialização da tela.
- **Solução:** foi criado level-strategy.js com a função classify(level, strategyName).

A implementação valida se o nível é numérico e está entre 0 e 100; depois retorna código, rótulo e cor para os estados crítico, atenção ou seguro.

var status = WaterLevelStrategy.classify(tank.level, "standard");

O código cliente recebe um objeto de status em vez de implementar todas as condições no local. A implementação fornecida contém uma estratégia chamada standard. Embora a estrutura permita adicionar outras estratégias, não se deve afirmar que existem várias políticas prontas: no código apresentado, apenas standard está definida.

4.2 Padrão 2 – Factory

- **Situação original:** números de reservatório, seletores HTML e valores padrão podiam estar espalhados pela página.

- **Solução:** tank-factory.js fornece WaterTankFactory.create(tankNumber, search), que valida o número e retorna um objeto com number, selector, statusSelector e level.

O nível é lido de parâmetros como tank1=65 e limitado ao intervalo de 0 a 100; quando ausente ou não numérico, usa o valor padrão 50.

var tank = WaterTankFactory.create(tankNumber, window.location.search);

A página deixa de montar individualmente os detalhes de configuração. A criação fica centralizada, favorecendo consistência e reutilização do componente.

4.3 Padrão 3 – Observer

- **Situação original:** a comunicação entre uma atualização e os componentes interessados não estava representada por um mecanismo explícito de inscrição.
- **Solução:** monitoring-observer.js implementa subscribe(listener) e notify(event).

O método notify percorre uma cópia da lista de ouvintes, reduzindo problemas caso a lista seja modificada durante a notificação.

observer.subscribe(function (event) {\n // atualizar a interface com os dados do evento\n});\nobserver.notify(event);

A classe Observer, isoladamente, só estabelece o mecanismo de publicação e inscrição. Para que o padrão tenha efeito real no sistema, a versão final precisa instanciar o observer, registrar pelo menos um listener e fazer a Facade ou outro produtor chamar notify. A equipe deve confirmar esses três pontos no código final e demonstrá-los durante a apresentação.

4.4 Padrão 4 – Facade

- **Situação original:** o cliente precisaria chamar Factory, Strategy e a notificação de forma coordenada.
- **Solução:** monitoring-facade.js recebe essas dependências e oferece initializeTank(tankNumber).

O método cria a configuração, classifica o nível e emite um evento com o número do reservatório, nível, seletores e status.

var tank = this.factory.create(tankNumber, this.search);\nvar status = this.strategy.classify(tank.level, "standard");\nthis.observer.notify({ tankNumber: tank.number, level: tank.level, status: status });

A Facade só está integrada se home.html carregar monitoring-facade.js, criar uma instância com as dependências corretas, registrar o listener responsável por atualizar a interface e chamar initializeTank para os reservatórios. O trecho da Facade fornecido define a coordenação, mas não comprova, por si só, que essa integração já existe na página final.

5. Análise dos Resultados

A avaliação abaixo é qualitativa e se baseia nos componentes compartilhados. Não foram fornecidos resultados de uma suíte automatizada nem métricas de linhas, complexidade ou cobertura; portanto, os benefícios são apresentados como efeitos estruturais observáveis ou esperados, e não como medições quantitativas.

Aspecto	Antes da refatoração	Depois da refatoração
Acoplamento	A página concentrava configurações, regras e atualização visual.	As responsabilidades estão distribuídas em componentes com funções específicas.
Coesão	O bloco de inicialização podia realizar várias tarefas diferentes.	Strategy classifica; Factory configura; Observer notifica; Facade coordena.
Extensibilidade	Nova regra ou forma de configuração poderia exigir mudanças no código da página.	É possível ampliar a estratégia ou a criação alterando o componente responsável.
Reutilização	A lógica embutida na página era mais difícil de reaproveitar isoladamente.	Os módulos podem ser chamados por outros clientes, desde que suas dependências sejam fornecidas.
Manutenção	As regras e detalhes de interface estavam próximos.	A separação facilita localizar a responsabilidade a modificar.
Duplicação	Havia risco de repetir condições e seletores para cada reservatório.	Factory centraliza a montagem da configuração; É necessário verificar o código final para medir a redução real.

5.1 Validação funcional recomendada

Executar o projeto a partir da pasta src com python3 -m http.server 8000 e abrir http://localhost:8000/home.html. Para validar a leitura de parâmetros, testar, por exemplo, ?tank1=20&tank2=55&tank3=72&tank4=95 e conferir se os valores e estados exibidos correspondem às regras definidas. Também testar nível ausente, valor não numérico e valores abaixo de 0 ou acima de 100.

Para Strategy, verificar que 20 e 95 são classificados como críticos, 55 como atenção e 72 como seguro, de acordo com os limites no código fornecido. Para Factory, conferir os valores padrão e a validação do número do reservatório. Para Observer, confirmar por console ou teste que um listener inscrito recebe o evento. Para Facade, confirmar que a página inicializa os reservatórios por initializeTank e que o listener atualiza os elementos da interface.

Registrar a data, o navegador, os parâmetros testados, o resultado observado e capturas de tela. Não declarar que os testes foram aprovados sem executar esses passos na versão final. O código compartilhado mostra validação de nível na Strategy e limitação de nível na Factory; não demonstra, por si só, que todos os cenários de integração já foram executados.

6. Conclusão

A refatoração do Water Monitoring System organiza quatro responsabilidades com padrões GoF complementares. Strategy isola a classificação dos níveis, Factory centraliza a configuração dos reservatórios, Observer introduz comunicação por inscrição e notificação, e Facade oferece um ponto de entrada de alto nível para coordenar o fluxo.

A principal contribuição é tornar o código mais organizado e facilitar a localização de mudanças. A separação cria condições para reutilizar componentes, testar regras isoladamente e ampliar comportamentos sem concentrar todas as alterações na página principal. A comparação qualitativa indica benefícios potenciais para coesão, acoplamento, manutenção e extensibilidade.

Entretanto, padrões não devem ser considerados concluídos apenas porque seus arquivos existem. Para demonstrar o resultado, é necessário confirmar que a página utiliza a Facade, que o Observer possui ouvintes inscritos e que os eventos atualizam a interface. Também é necessário executar os testes funcionais e guardar evidências.

Conclui-se que o trabalho atende ao propósito acadêmico de relacionar problemas concretos a padrões de projeto, desde que a equipe finalize a validação da integração e registre o commit entregue. Como continuidade, recomenda-se automatizar os testes de Strategy, Factory, Observer e Facade, documentar os resultados e comparar quantitativamente a versão original com a refatorada.

7. Referências

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1994.

MARTIN, Robert C. Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River: Prentice Hall, 2008.

SHAHDEO, Vinit. Water Monitoring System. GitHub. Disponível em: https://github.com/vinitshahdeo/water-monitoring-system. Acesso em: 13 out. 2026.

SHAHDEO, Vinit. Water Monitoring System — demonstração web. Disponível em: https://vinitshahdeo.github.io/Water-Monitoring-System/. Acesso em: 13 out. 2026.

ORACLE. JavaScript Guide / documentação da linguagem JavaScript. Disponível em: https://developer.mozilla.org/en-US/docs/Web/JavaScript. Acesso em: 13 out. 2026.

MOZILLA DEVELOPER NETWORK. JavaScript Guide. Disponível em: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide. Acesso em: 13 out. 2026.